

DATA ENGINEERING

# Automated Violence Detection for Film Classification

Group 4

김유림, 백지현, 이재린, 이채연, 장서연, 한예원

# INDEX

|                   |    |
|-------------------|----|
| <b>Background</b> | 01 |
|-------------------|----|

|                 |    |
|-----------------|----|
| <b>Pipeline</b> | 02 |
|-----------------|----|

|                              |    |
|------------------------------|----|
| <b>Data &amp; Annotation</b> | 03 |
|------------------------------|----|

|                 |    |
|-----------------|----|
| <b>Modeling</b> | 04 |
|-----------------|----|

|             |    |
|-------------|----|
| <b>Demo</b> | 05 |
|-------------|----|

|                   |    |
|-------------------|----|
| <b>Conclusion</b> | 06 |
|-------------------|----|

|                 |    |
|-----------------|----|
| <b>Appendix</b> | 07 |
|-----------------|----|

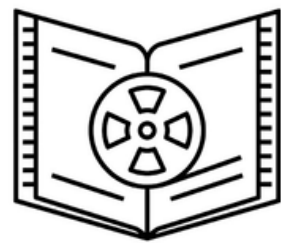
01

# Background

01

# Film Rating System

Korea Media Rating Board (KM RB) Classification Criteria



Theme



Sexuality



Violence



Language



Horror



Drugs



Imitation Risk

OTT surge: 15–20 films/week per reviewer, months of delay → **automation needed**





# Single-frame detection vs. Violence

## Frame-level

Smoking· Drinking· Nudity

### Detection:

#### WHAT

Detection reduces to whether the object is present - cigarettes, bottles, exposed skin, weapons.

#### HOW

Object detection + bounding boxes capture it directly from one frame

## Sequence-level

Violence

### Detection:

#### WHAT

Not what object appears, but the action unfolds - fighting, beating, group violence

#### HOW

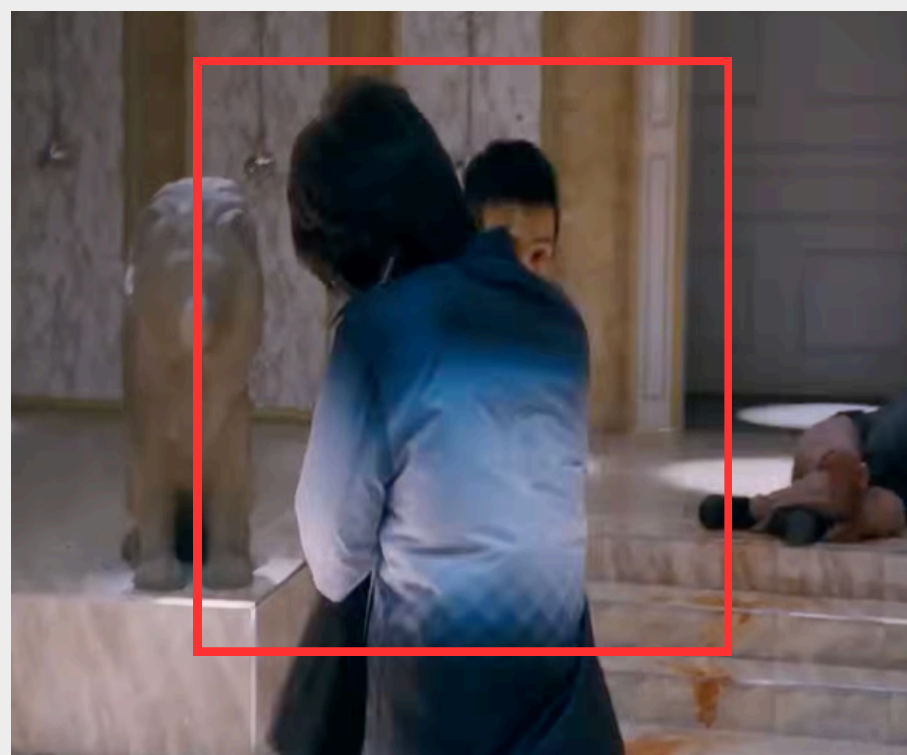
Consider motion, causality and context across frames



01

# Why a violence classifier is needed

One frame tells you nothing → Violence is defined by what happens next



Violence?

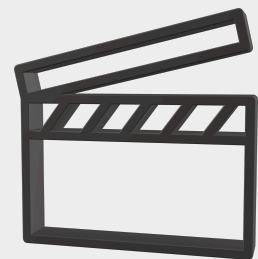


Now it's clear

A single frame & bounding box cannot capture context  
→ We need a violence classifier that understands sequential flow

02

# Pipeline



02

# Main Pipeline

---

## 1. Data Collection

---

Weak labels from movie review videos

## 2. Sequence-Level Annotation

---

Capturing violence over time

## 3. Data Storage

---

From collaboration to efficient retrieval

## 4. Preprocessing

---

Preparing clips for temporal modeling

## 5. Modeling & Evaluation

---

Finding the best temporal model

## 6. Demo

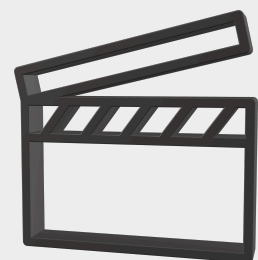
---

Violence interval prediction

03

# Data & Annotation





# Data: Why Movie Review Videos?

---

## Original Movies

---

- Copyright restrictions
- Need to watch entire films manually
- Violent scenes are sparse
- High annotation cost

## Movie Review Videos

---

- Publicly accessible on YouTube
- Often edited to highlight key scenes
- Higher density of violent events
- More efficient dataset construction

**From weak labels to a reliable training dataset.**



# Data Collection

Korean Movie Review Videos on YouTube  
(20-40min)



yt-dlp Download



To increase the likelihood of capturing violent events, we collected Korean movie review videos from genres such as action, crime, noir, and thriller.

```
명령 프롬프트
C:\Users\urimh>pip install yt-dlp
Requirement already satisfied: yt-dlp in c:\users\urimh\appdata\local\python\pythoncore-3.14-64\lib\site-packages (2026.3.17)

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: C:\Users\urimh\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

C:\Users\urimh>yt-dlp -o "C:\Users\urimh\Desktop\학교\3-1\데 엔\proj2\Witch.%(ext)s" "https://www.youtube.com/watch?v=_EWIe0PjQ6g"
[youtube] Extracting URL: https://www.youtube.com/watch?v=_EWIe0PjQ6g
[youtube] _EWIe0PjQ6g: Downloading webpage
```

YouTube videos were automatically downloaded using yt-dlp.

**Dataset Overview: 62 videos / 174,253 extracted frames**





# Why Sequence-Level Annotation?

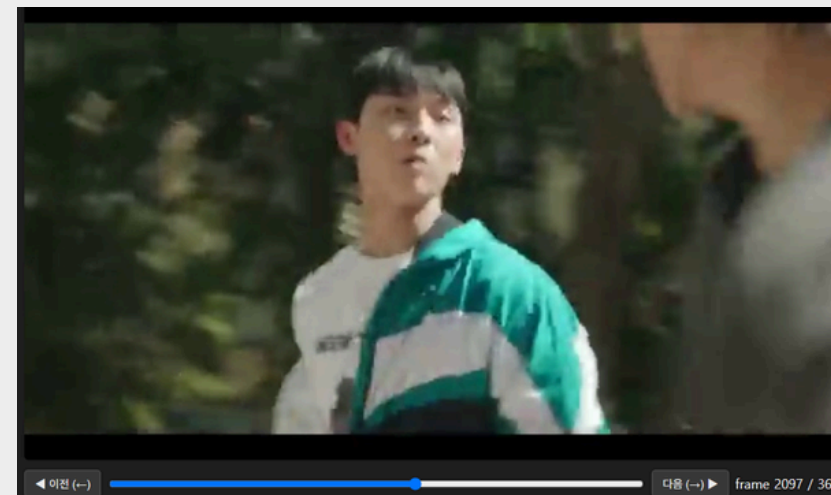
## Bounding Box Annotation



Identifies objects within a single frame.  
Cannot determine whether the action is violent from a single frame.



## Sequence Level Annotation



Captures motion, interaction, temporal context

**Determines violence based on the sequence of events.**





# Annotate with Custom Annotation Tool

To efficiently annotate long review videos while preserving temporal context, we developed a custom sequence-level annotation tool.

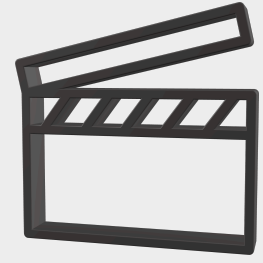
## Frame Extraction

- Videos sampled at 2 frames per second (2 fps)
- Reduced redundancy and annotation workload
- Preserved temporal information

## Features

- Keyboard-based annotation
- Start/end frame selection
- Sequence-level labeling
- TXT export





# Annotate with Custom Annotation Tool

## Annotation Workflow

- Violence : Explicit physical aggression
- Neg\_hard : non-violent scenes resembling violence (e.g., play fighting, training)
- Neg\_easy : Clearly non-violent scenes

**Only violence and hard-negative segments were manually annotated, while the remaining intervals were automatically assigned as neg\_easy.**



Annotators marked the start and end timestamps (start\_sec-end\_sec)  
of violence-related events.



# Data Storage Architecture

---

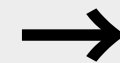
58 annotated review videos, about 29,000 extracted frames



**Hugging Face**

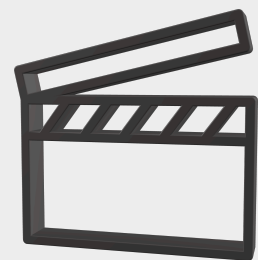
- Shared annotation files across team members
- Version control for dataset updates
- Easy access to metadata

Dataset Growth  
(~ 170,000Frames)



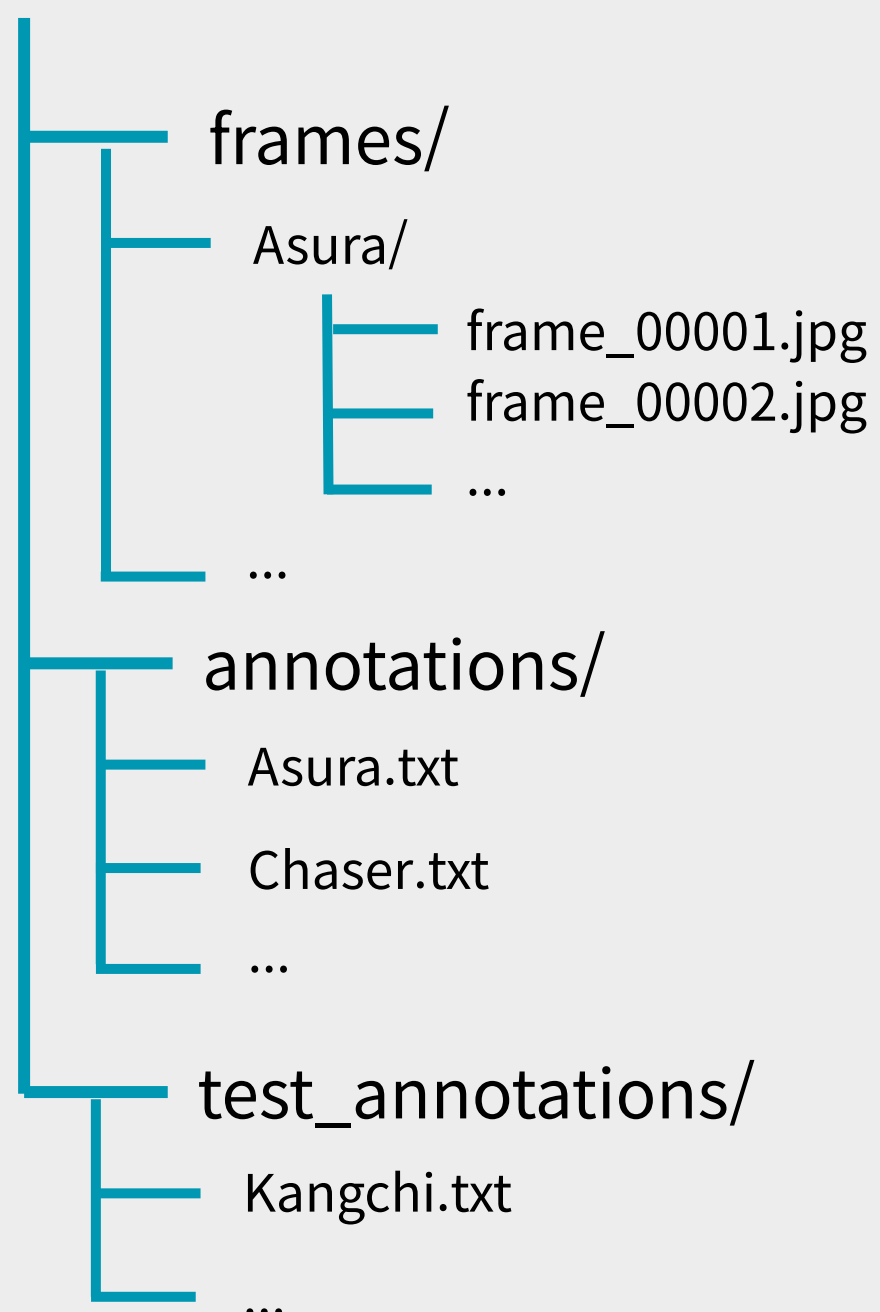
- Stored large media assets efficiently
- S3-compatible object storage
- Managed extracted frames and video files

**Hugging Face for collaboration, MinIO for efficient training.**



# Dataset Organization

**dataset/**



## Frames

- Extracted at 2 fps
- Stored by movie ID
- Used as inputs for annotation and preprocessing

## Annotations

- Sequence-level labels
- Annotation Format  
[movie\_id, scene\_num, start\_frame, end\_frame, label]

## Test Data

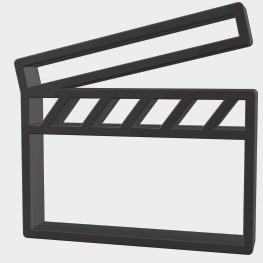
- Maintained separately for final evaluation
- Used only during testing

04

# Modeling

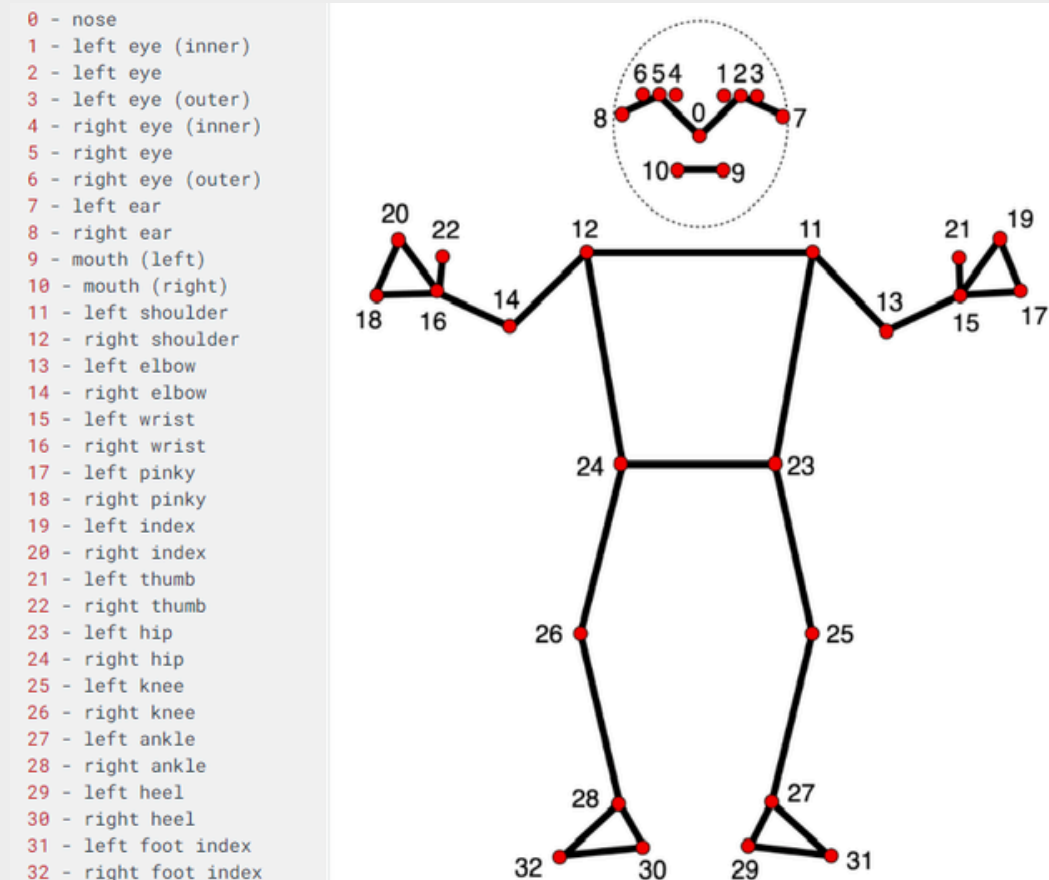
- 1. Pose Detection & Optical Flow: Attempt & Limitations**
- 2. CNN Feature Extraction**
- 3. Sequential Model Experiments**
- 4. Final Model & Performance Analysis**





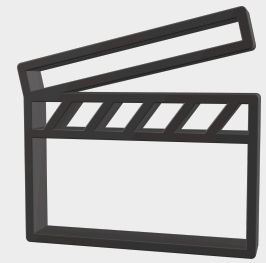
# Implementation of Pose Detection Pipeline

## MediaPipe



Why MediaPipe? Violence detection requires understanding human body movements

- MediaPipe extracts **33 joint coordinates** per frame
- Lightweight: **runs on CPU** without GPU



# Implementation of Pose Detection Pipeline

## Pipeline of MediaPipe

1. frame



2. MediaPipe

```
def extract_keypoints(image_path):  
    image = mp.Image.create_from_file(image_path)  
    result = detector.detect(image)  
    if result.pose_landmarks:  
        kp = np.array([[lm.x, lm.y] for lm in result.pose_landmarks[0]])  
        return kp # (33, 2)  
    return np.zeros((33, 2))
```

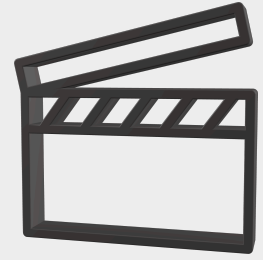
3. get (33, 2) keypoints



4. visualize and store it for LSTM







# Failure Case of Pose Detection



## Close-up shots

full body not visible  
→ keypoints missing



## Motion blur

camera shake during action  
→ unstable detection



## Dark scenes

insufficient lighting  
→ detection failed



# Motion Detection - Optical flow

## What is optical flow?



Tracks how each pixel moves between consecutive frames — capturing the direction and speed of motion.

- Reading the visualization:
  - Long arrows → fast movement (players)
  - Short/no arrows → stationary regions (background)
  - In violence scenes → large, rapid arrows around fists or kicks



# Motion Detection - Optical flow

## Optical Flow Implementation

```
def compute_flow_magnitude(gray1, gray2) -> float:
    # ✅ 리사이즈 후 flow 계산
    g1 = cv2.resize(gray1, (320, 240))
    g2 = cv2.resize(gray2, (320, 240))
    flow = cv2.calcOpticalFlowFarneback(
        g1, g2, None,
        pyr_scale=0.5,
        levels=2,      # 3 → 2
        winsize=11,    # 15 → 11
        iterations=2,  # 3 → 2
        poly_n=5,
        poly_sigma=1.2,
        flags=0
    )
    return float(np.sqrt(flow[..., 0]**2 + flow[..., 1]**2).mean())
```

Farneback algorithm extracts dense flow vectors across all pixels, then compresses to a single magnitude value per frame pair

To reduce computation time:

- Resized frames to  $320 \times 240$  before flow calculation  $\rightarrow \sim 27\times$  fewer pixels to process
- Reduced pyramid levels  $3 \rightarrow 2$ , window size  $15 \rightarrow 11$ , iterations  $3 \rightarrow 2$   $\rightarrow \sim 8\times$  overall speedup
- Compressed all pixel-level flow vectors into a single magnitude value per frame pair



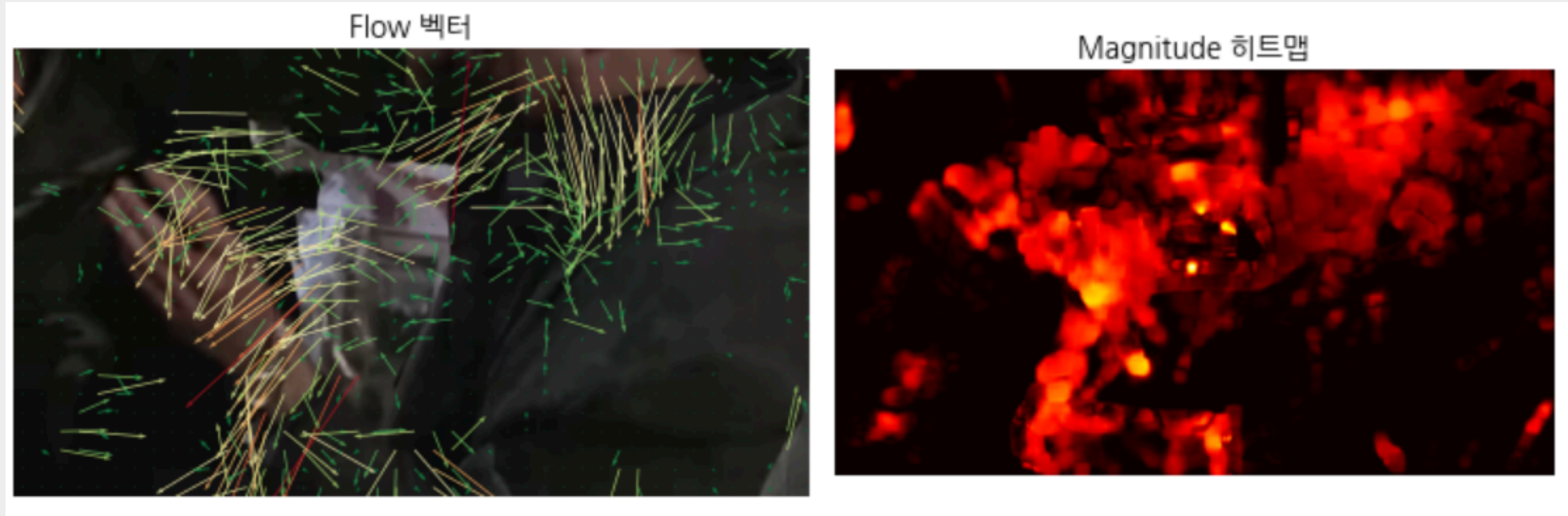


04-1

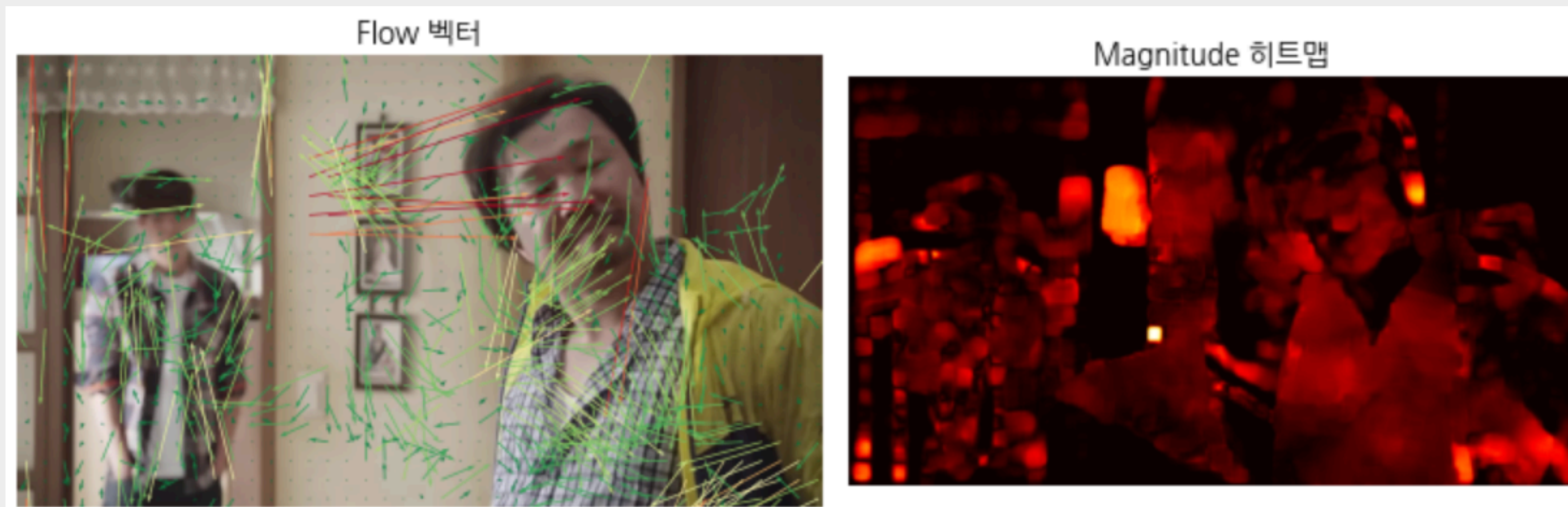
# Motion Detection - Optical flow

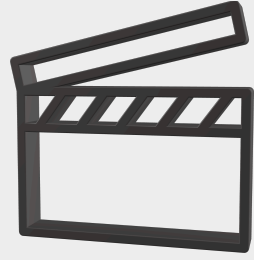
## Visualization

- Class : violence



- Class : neg\_easy





# Why CNN

## Raw frames can't go directly into sequence models

$224 \times 224 \times 3 = 150,528$  dimension per frame

→ too high-dimensional !!

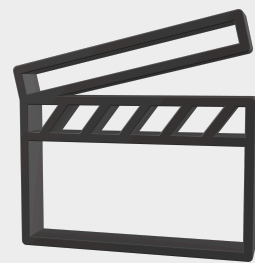
## CNN works where Mediapipe failed

Extracts visual features directly from pixels

→ robust to motion blur, dark scenes, and close-up shots

## ResNet50 - pretrained on ImageNet

Rich visual features already learned - used as frozen feature extractor, no fine-tuning needed



04-2

# ResNet50 image feature extraction

Remove final FC layer → 2048-dim feature vector

```
resnet = models.resnet50(weights='IMAGENET1K_V1')  
resnet = torch.nn.Sequential(*list(resnet.children())[:-1])  
resnet.eval().to(device)
```

- Used as frozen feature extractor (weights not updated)
- 4 frames → (4, 2048) : Extracted CNN features for each clip
- X.npy (N, 4, 2048) → Precomputed once and saved
- Input to sequence models



**Output**

- clips\_jaelin\_test\_movie\_ids.npy
- clips\_jaelin\_test\_X.npy
- clips\_jaelin\_test\_y.npy





# Clip Construction & Normalization

## Clip Construction

```
for (start, end, label) in scenes:
    frames = list(range(start, end+1))
    for i in range(0, len(frames) - clip_len + 1, stride):
        clip_frames = frames[i:i+clip_len]
        clip_feats = []
        for f in clip_frames:
            img_path = f'{frames_dir}/frame_{f:06d}.jpg'
            if os.path.exists(img_path):
                clip_feats.append(extract_cnn_feature(img_path))
            else:
                clip_feats.append(np.zeros(2048))
        if len(clip_feats) == clip_len:
            if label == 'violence':
                vio_clips.append(clip_feats)
                vio_labels.append('violence')
            else:
                neg_clips.append(clip_feats)
                neg_labels.append('neg_easy') # neg_hard도 neg_easy로 저장
```

**clip\_len=4**

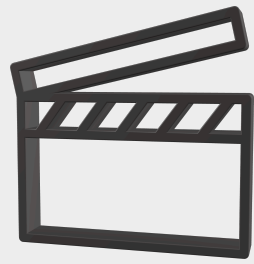
- 4 frames per clip = 2 seconds (at 2fps)
- Shape: (4, 2048) → temporal sequence
- Why 4 frames: minimum unit to capture one action

## Normalization

```
# 영화별 정규화
X_norm = X.copy()
for movie in set(movie_ids):
    idx = np.where(movie_ids == movie)[0]
    if len(idx) > 1:
        orig_shape = X[idx].shape
        flat = X[idx].reshape(len(idx), -1)
        flat = StandardScaler().fit_transform(flat)
        X_norm[idx] = flat.reshape(orig_shape)

print('정규화 완료')
```

- Each movie has different lighting/color characteristics
  - Apply StandardScaler per movie
- Prevents domain shift between movies



04-3

# Experiment Design

## Load .npy files and append all

```
TRAIN_PATH = '/content/drive/MyDrive/ColabNotebooks/dataEng

X_all, y_all, movie_all = [], [], []
for x_path in sorted(Path(TRAIN_PATH).glob('*_X.npy')):
    name = x_path.stem.replace('_X', '')
    y_path = x_path.parent / f'{name}_y.npy'
    m_path = x_path.parent / f'{name}_movie_ids.npy'
    X_all.append(np.load(x_path))
    y_all.append(np.load(y_path))
    movie_all.append(np.load(m_path))

X = np.concatenate(X_all).astype(np.float32)
y_str = np.concatenate(y_all)
movie_ids = np.concatenate(movie_all)

LABEL_MAP = {'neg_easy': 0, 'violence': 1}
y = np.array([LABEL_MAP[l] for l in y_str], dtype=np.int64)
```

## Movie-level Train/Val/Test Split with movie ID

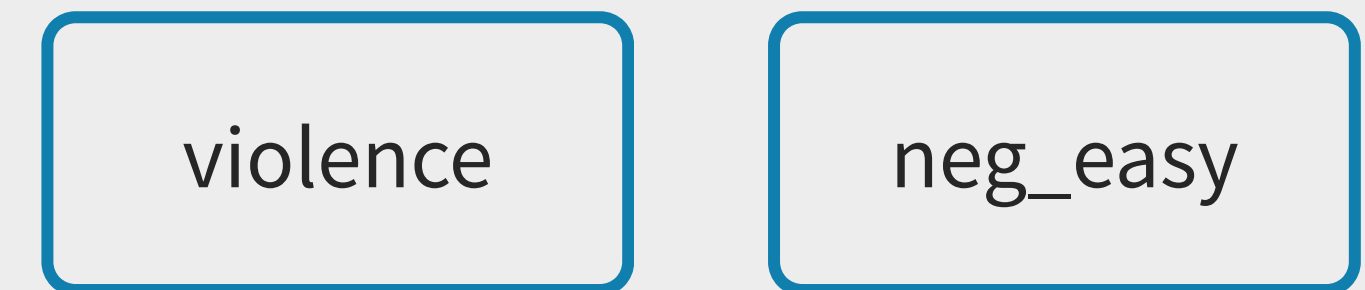
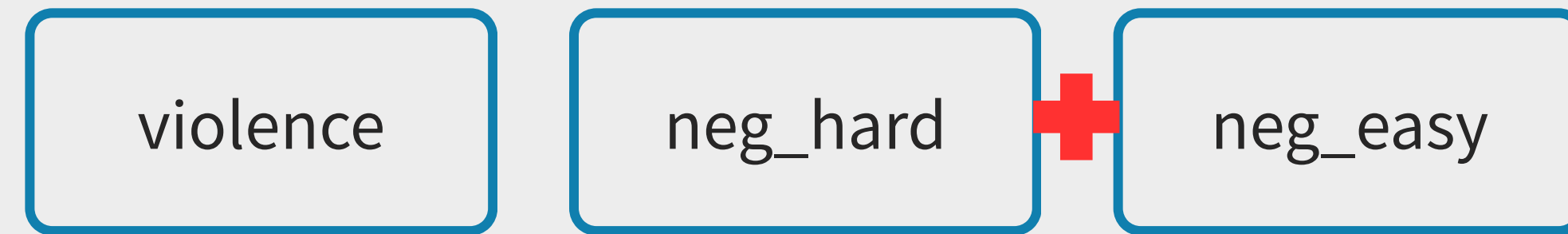
```
train_idx = np.where(np.isin(movie_ids, train_movies))[0]
val_idx = np.where(np.isin(movie_ids, val_movies))[0]

X_train, y_train = X_norm[train_idx], y[train_idx]
X_val, y_val = X_norm[val_idx], y[val_idx]

print(f'train: {len(X_train)}개 ({len(train_movies)}개 영화)')
print(f'val: {len(X_val)}개 ({len(val_movies)}개 영화)')

train: 27513개 (49개 영화)
val: 6205개 (13개 영화)
```

neg\_hard (ambiguous violence)  
merged into neg\_easy





# Baseline: Traditional ML

## Logistic Regression

```
.. 피쳐 shape | train: (25998, 4096) | val: (7720, 4096) | test: (5195, 4096)
[LogReg] VAL: acc=0.6921 | violence F1=0.6573
[LogReg] TEST: acc=0.6830 | violence F1=0.6107
--- TEST classification report ---
              precision    recall  f1-score   support

neg_easy      0.74      0.72      0.73      3116
violence      0.60      0.62      0.61      2079

accuracy              0.68      5195
macro avg      0.67      0.67      0.67      5195
weighted avg    0.68      0.68      0.68      5195
```

**Test F1 (violence): 0.61**

**Test Accuracy: 0.68**

## Linear SVM

```
[LinearSVM] VAL: acc=0.6907 | violence F1=0.6556
[LinearSVM] TEST: acc=0.6810 | violence F1=0.6086
--- TEST classification report ---
              precision    recall  f1-score   support

neg_easy      0.74      0.72      0.73      3116
violence      0.60      0.62      0.61      2079

accuracy              0.68      5195
macro avg      0.67      0.67      0.67      5195
weighted avg    0.68      0.68      0.68      5195
```

**Test F1 (violence): 0.608**

**Test Accuracy: 0.68**

### Limitations of Traditional ML:

- Treats each clip as a flat vector → frame order is ignored, temporal structure is lost
- Suitable for static features, not sequential patterns

**→ Sequential deep learning models needed**



04-3

# LSTM

## From Static to Sequential

```
class ViolenceLSTM(nn.Module):
    def __init__(self, input_size=2048, hidden_size=256, num_layers=2, dropout=0.3):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True, dropout=dropout)
        self.fc = nn.Linear(hidden_size, 2)
    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

model = ViolenceLSTM().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- Input: (batch, 4, 2048) temporal sequence
- Hidden size: 256, Layers: 2, Dropout: 0.3
- Output: violence / neg\_easy

## Result

Test F1 (violence): 0.62  
Test Accuracy: 0.71

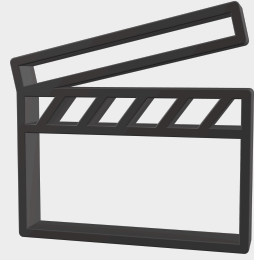
+3%p accuracy gain!!

```
--- Test classification report ---
              precision    recall  f1-score   support

   neg_easy      0.74      0.79      0.77      3116
   violence      0.65      0.59      0.62      2079

 accuracy      0.71      0.71      0.71      5195
  macro avg      0.70      0.69      0.69      5195
 weighted avg      0.71      0.71      0.71      5195
```





# Transformer Encoder

## Advantage over LSTM

```
class ViolenceTransformer(nn.Module):
    def __init__(self, input_size=2048, nhead=8, num_layers=2, dropout=0.3):
        super().__init__()
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=input_size,
            nhead=nhead,
            dim_feedforward=512,
            dropout=dropout,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.fc = nn.Linear(input_size, 2)

    def forward(self, x):
        out = self.transformer(x)
        return self.fc(out[:, -1, :])
```

### Transformer Encoder

**-Self-attention:** Learns which frames matter most

**-Positional encoding:** Captures frame order without sequential processing

## Result

```
--- Transformer Test classification report ---
              precision    recall  f1-score   support

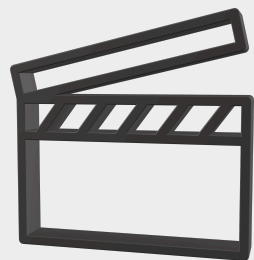
   neg_easy         0.74         0.81         0.78         3116
   violence         0.67         0.58         0.62         2079

   accuracy                   0.72         5195
  macro avg         0.71         0.70         0.70         5195
 weighted avg         0.72         0.72         0.72         5195
```

Test F1 (violence): 0.62  
Test Accuracy: 0.72

**+4%p accuracy gain!!**





04-3

# Overfitting Issue & Mitigation

## Overfitting Issue

|            |                    |                  |                   |                 |
|------------|--------------------|------------------|-------------------|-----------------|
| Epoch 1/30 | Train Loss: 0.5135 | Val Loss: 0.6750 | Train Acc: 0.7497 | Val Acc: 0.6750 |
| Epoch 2/30 | Train Loss: 0.3897 | Val Loss: 0.6736 | Train Acc: 0.8261 | Val Acc: 0.6869 |
| Epoch 3/30 | Train Loss: 0.3057 | Val Loss: 0.7500 | Train Acc: 0.8687 | Val Acc: 0.6892 |



### Huge Gap!!

|             |                    |                  |                   |                 |
|-------------|--------------------|------------------|-------------------|-----------------|
| Epoch 26/30 | Train Loss: 0.0369 | Val Loss: 2.4001 | Train Acc: 0.9866 | Val Acc: 0.6783 |
| Epoch 27/30 | Train Loss: 0.0258 | Val Loss: 2.8297 | Train Acc: 0.9907 | Val Acc: 0.6773 |
| Epoch 28/30 | Train Loss: 0.0305 | Val Loss: 2.7524 | Train Acc: 0.9905 | Val Acc: 0.6791 |
| Epoch 29/30 | Train Loss: 0.0349 | Val Loss: 2.5016 | Train Acc: 0.9885 | Val Acc: 0.6591 |
| Epoch 30/30 | Train Loss: 0.0298 | Val Loss: 2.6950 | Train Acc: 0.9895 | Val Acc: 0.6728 |

Overfitting Detected!

- Original: Train Acc  $\approx$  0.99 vs Val Acc  $\approx$  0.66 (gap  $\approx$  0.33)
- Cause: movie-level domain difference  
+ limited training data

Result:

- Improved: Train Acc 0.79 vs Val Acc 0.73 (gap  $\approx$  0.06)



## Mitigation

```
dropout=0.5): # dropout 0.3 → 0.5
```



```
criterion = nn.CrossEntropyLoss()  
optimizer = torch.optim.Adam(model_wd.parameters(), lr=1e-3, weight_decay=1e-4)  
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=20)
```

- Dropout: 0.3  $\rightarrow$  0.5
- Weight Decay: 1e-4
- CosineAnnealing LR Scheduler
- Best model checkpoint saving



### Smaller Gap!!

|                               |                    |                  |                   |                 |
|-------------------------------|--------------------|------------------|-------------------|-----------------|
| ✓ Best 모델 저장 (val_acc=0.7370) |                    |                  |                   |                 |
| Epoch 25/30                   | Train Loss: 0.4381 | Val Loss: 0.6586 | Train Acc: 0.8014 | Val Acc: 0.7334 |
| Epoch 26/30                   | Train Loss: 0.4361 | Val Loss: 0.6575 | Train Acc: 0.8033 | Val Acc: 0.7341 |
| Epoch 27/30                   | Train Loss: 0.4413 | Val Loss: 0.6745 | Train Acc: 0.8028 | Val Acc: 0.7339 |
| Epoch 28/30                   | Train Loss: 0.4426 | Val Loss: 0.6907 | Train Acc: 0.8007 | Val Acc: 0.7384 |
| ✓ Best 모델 저장 (val_acc=0.7384) |                    |                  |                   |                 |
| Epoch 29/30                   | Train Loss: 0.4469 | Val Loss: 0.6768 | Train Acc: 0.7969 | Val Acc: 0.7293 |
| Epoch 30/30                   | Train Loss: 0.4544 | Val Loss: 0.6485 | Train Acc: 0.7908 | Val Acc: 0.7280 |



# Final Model Results

## Model Architecture and Train Recipe

```
class ViolenceTransformerFinal(nn.Module):
    def __init__(self, input_size=2048, nhead=8, num_layers=2, dropout=0.5): # dropout 0.3 → 0.5
        super().__init__()
        self.layers = nn.ModuleList([
            nn.TransformerEncoderLayer(
                d_model=input_size,
                nhead=nhead,
                dim_feedforward=512,
                dropout=dropout,
                batch_first=True
            ) for _ in range(num_layers)
        ])
        self.fc = nn.Linear(input_size, 2)

    def forward(self, x, return_attn=False):
        attn_weights = []
        out = x
        for layer in self.layers:
            attn_out, attn_w = layer.self_attn(out, out, out, average_attn_weights=True)
            attn_weights.append(attn_w.detach().cpu())
            out = layer(out)
        logits = self.fc(out[:, -1, :])
        if return_attn:
            return logits, attn_weights
        return logits

model_wd = ViolenceTransformerFinal().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_wd.parameters(), lr=1e-3, weight_decay=1e-4) # weight_decay 추가
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=20)
```

Final Model: Transformer Encoder

Dropout: 0.5 | Weight Decay: 1e-4 | CosineAnnealing Scheduler

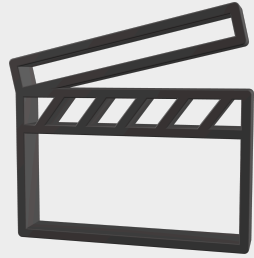
Train Accuracy: 0.79 | Val Accuracy: 0.73

Test F1 (violence): 0.65 | Test Accuracy: 0.74

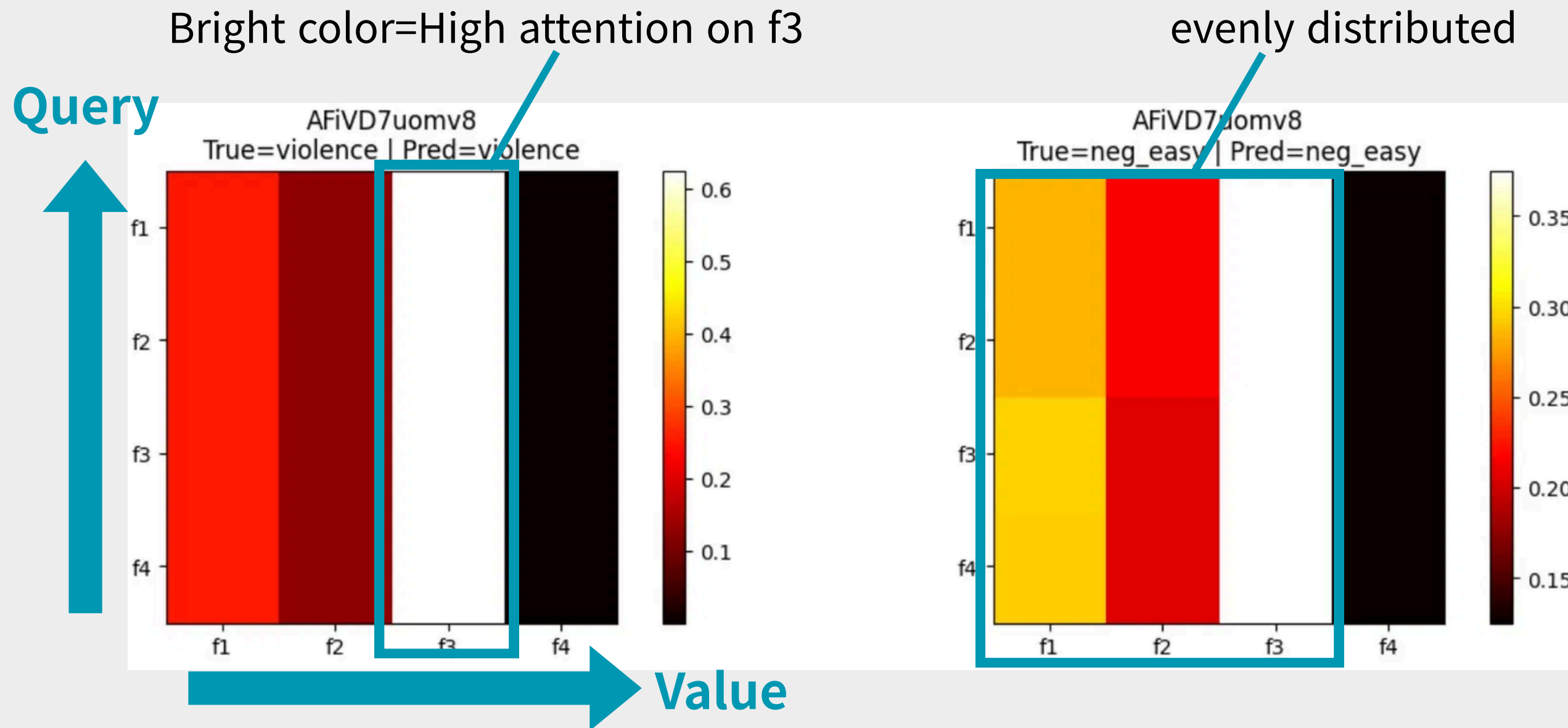
**+6%p accuracy gain!!**

--- Test classification report ---

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| neg_easy     | 0.76      | 0.83   | 0.79     | 3116    |
| violence     | 0.70      | 0.61   | 0.65     | 2079    |
| accuracy     |           |        | 0.74     | 5195    |
| macro avg    | 0.73      | 0.72   | 0.72     | 5195    |
| weighted avg | 0.74      | 0.74   | 0.74     | 5195    |



# Attention Visualization: Violence vs Neg\_easy



Violence (correctly predicted)

- High attention concentrated on f3
- f4 receives near-zero attention
- Model focuses on a specific moment

Neg\_easy (correctly predicted)

- Attention evenly distributed across f1, f2, f3
- No dominant frame stands out



# Model Comparison

| Model                               | Test F1(violence) | Test Acc |
|-------------------------------------|-------------------|----------|
| Logistic Regression<br>/ Linear SVM | 0.61              | 0.68     |
| LSTM                                | 0.62              | 0.71     |
| TCN                                 | 0.57              | 0.69     |
| BiLSTM                              | 0.59              | 0.71     |
| Transformer                         | 0.62              | 0.72     |

05

# Demo



# Demo App

## Video Uploader

- Video name
- Add video

## Video Player

- Violence progress bar

### ● 영화 폭력 구간 탐지 데모

영상 업로드 → 폭력 구간(start~end) 타임라인. ResNet50 피쳐 + Transformer (test Acc 0.72). 학습/test 71편 외 영상으로 데모하세요.

영상 업로드

범죄도시.mp4 51.7MB

+

영상 길이  
13:59.0

클립 수  
838

폭력 구간  
28개

영상 내 폭력 클립 비율  
35%

▲ '영상 내 폭력 클립 비율'은 이 영상 안에서의 상대값입니다. 피쳐가 영상 단위로 정규화되므로 서로 다른 영상끼리 이 수치를 비교해 '어느 영화가 더 폭력적인가'를 판단하면 안 됩니다 (영화 단위 폭력 강도는 이 시스템의 유효한 출력이 아님).



▶ 재생 0:00 / 14:00

● 폭력 구간 (바를 클릭하면 이동)

## Metric Cards

- Video length
- Clip count
- Violence segments
- In-video violence clip ratio

# Demo App

## Violence Timeline Chart

- per-clip violence probability
- detected violence segments

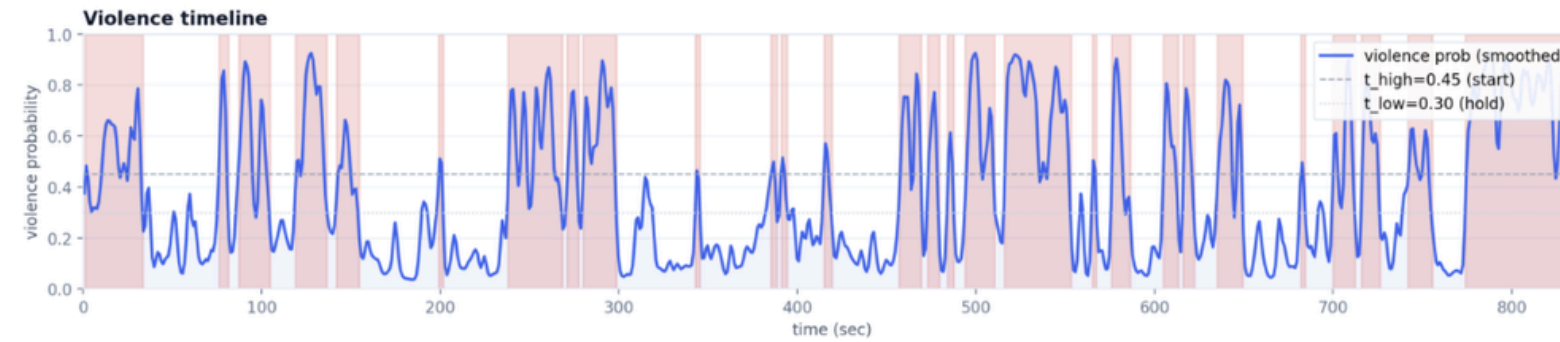
⋮

## Gaussian smoothing

: strong neighboring clips pull up short dips (neighbor influence)

## Hysteresis dual-threshold

: start a segment at  $t_{high}$ , keep it alive until it drops below  $t_{low}$



## 탐지된 폭력 구간

| 구간                | 길이(초) | 최고확률 |
|-------------------|-------|------|
| 00:01.0 ~ 00:33.5 | 32.5  | 0.79 |
| 01:16.0 ~ 01:21.5 | 5.5   | 0.86 |
| 01:27.0 ~ 01:44.5 | 17.5  | 0.89 |
| 01:59.0 ~ 02:16.5 | 17.5  | 0.92 |
| 02:22.0 ~ 02:34.5 | 12.5  | 0.66 |
| 03:19.0 ~ 03:21.5 | 2.5   | 0.51 |
| 03:58.0 ~ 04:28.5 | 30.5  | 0.87 |
| 04:31.0 ~ 04:37.5 | 6.5   | 0.78 |
| 04:40.0 ~ 04:58.5 | 18.5  | 0.89 |
| 05:43.0 ~ 05:45.5 | 2.5   | 0.46 |
| 06:25.0 ~ 06:28.5 | 3.5   | 0.50 |
| 06:31.0 ~ 06:34.5 | 3.5   | 0.51 |
| 06:55.0 ~ 06:59.5 | 4.5   | 0.57 |
| 07:37.0 ~ 07:49.5 | 12.5  | 0.84 |

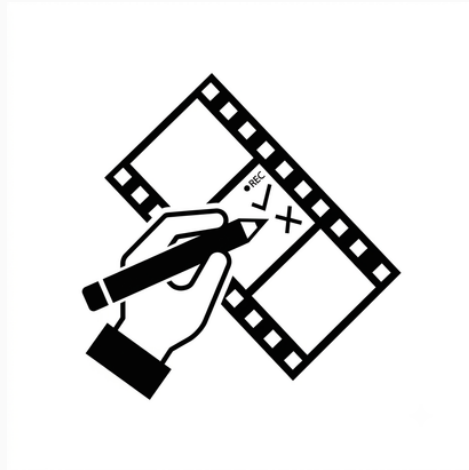
## Detected Segment Table

- the time range
- duration
- peak probability

06

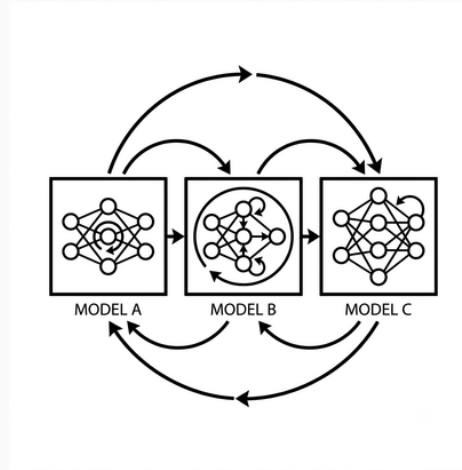
# Conclusion

# What we achieved



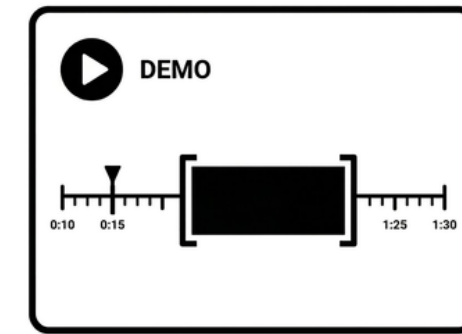
## Data & Annotation

Built out own dataset  
& annotation tool from scratch



## Model comparison

Iteratively trained and compared  
multiple models



## Working demo

New video in → violence  
segments [start, end] out

# Limitation & Future Work

## What We Learned

### Model plateaued

RGB features miss motion  
→ Appearance alone isn't enough

### Richer data needed

RGB features miss motion  
→ Appearance alone isn't enough

### Annotation consistency

Human-in-the-loop annotation introduces  
subjectivity



## Where We Go Next

### End-to-End fine-tuning

Fine-tune backbone on violence data  
directly or adopt a video-native model

### Move beyond review clips

Full films provide cleaner, higher-quality  
violence segments

### Cross-check annotations

Multi-annotator cross-checking can  
reduce subjectivity



07

# Appendix



# Motion Detection - Optical flow

## Prediction

| clip_length | Validation F1(violence) | Validation Acc |
|-------------|-------------------------|----------------|
| 2           | 0.5756                  | 0.6000         |
| 4           | 0.6240                  | 0.6281         |
| 8           | 0.6565                  | 0.6586         |

- Longer clip length captured more temporal context → gradual improvement in F1 score
- However, performance remained insufficient for practical use



# Additional Models: TCN, BiLSTM

## TCN

```
class ViolenceTCN(nn.Module):
    def __init__(self, input_size=2048, num_channels=[128, 128, 256], dropout=0.3):
        super().__init__()
        self.tcn = TCN(
            num_inputs=input_size,
            num_channels=num_channels,
            kernel_size=2,
            dropout=dropout,
            input_shape='NLC'
        )
        self.fc = nn.Linear(num_channels[-1], 2)

    def forward(self, x):
        out = self.tcn(x)
        return self.fc(out[:, -1, :])
```

```
--- TCN Test classification report ---
              precision    recall  f1-score   support

 neg_easy         0.71         0.81         0.76         3116
 violence         0.64         0.51         0.57         2079

 accuracy                             0.69         5195
 macro avg         0.68         0.66         0.66         5195
 weighted avg         0.68         0.69         0.68         5195
```

TCN (Temporal Convolutional Network)

- CNN-based, parallel processing
- Test F1: 0.57 | Test acc : 0.69

## BiLSTM (Bidirectional LSTM)

```
class ViolenceBiLSTM(nn.Module):
    def __init__(self, input_size=2048, hidden_size=256, num_layers=2, dropout=0.3):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size, hidden_size, num_layers,
            batch_first=True, dropout=dropout,
            bidirectional=True # 이게 핵심
        )
        self.fc = nn.Linear(hidden_size * 2, 2) # bidirectional이라 *2

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])
```

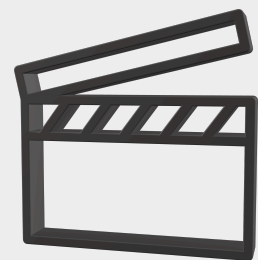
```
--- BiLSTM Test classification report ---
              precision    recall  f1-score   support

 neg_easy         0.73         0.83         0.77         3116
 violence         0.67         0.53         0.59         2079

 accuracy                             0.71         5195
 macro avg         0.70         0.68         0.68         5195
 weighted avg         0.70         0.71         0.70         5195
```

BiLSTM (Bidirectional LSTM)

- Captures both forward and backward context
- Test F1: 0.59 | Test acc : 0.71



# Per-Movie Performance

Upper vs. Lower cluster differ significantly across all three metrics (2-proportion z-test)

## | Per-Movie F1 (Transformer + Viz)

| movie       | n_clips | f1_violence | acc      | precision_v | recall_v |
|-------------|---------|-------------|----------|-------------|----------|
| Extremejob  | 377     | 0.706714    | 0.779841 | 0.757576    | 0.662252 |
| Villains2   | 245     | 0.687500    | 0.755102 | 0.702128    | 0.673469 |
| PgDp_1RvZfg | 712     | 0.682482    | 0.755618 | 0.711027    | 0.656140 |
| Insideme    | 557     | 0.671498    | 0.755835 | 0.727749    | 0.623318 |
| _8CiLAlnaKk | 440     | 0.664740    | 0.736364 | 0.676471    | 0.653409 |
| Witch       | 912     | 0.644412    | 0.731360 | 0.685185    | 0.608219 |
| J9oPKXbiG14 | 345     | 0.553846    | 0.663768 | 0.590164    | 0.521739 |
| Thewild     | 1107    | 0.551471    | 0.669377 | 0.603217    | 0.507901 |
| Kangchi     | 500     | 0.533708    | 0.668000 | 0.608974    | 0.475000 |

## Statistical Findings -Significant F1 Gap:

Upper cluster F1 = 0.671

Lower cluster F1 = 0.547

→ Difference 0.124

## <2-proportion z-test>

Recall  $z = 6.13$ ,  $p = 8.6 \times 10^{-10}$

Precision  $z = 4.52$ ,  $p = 6.1 \times 10^{-6}$

Accuracy  $z = 6.28$ ,  $p = 3.3 \times 10^{-10}$

## Recall drop drives the gap

Recall: 0.639 → 0.502 ( $\Delta$  0.137)

Precision: 0.706 → 0.602 ( $\Delta$  0.104)

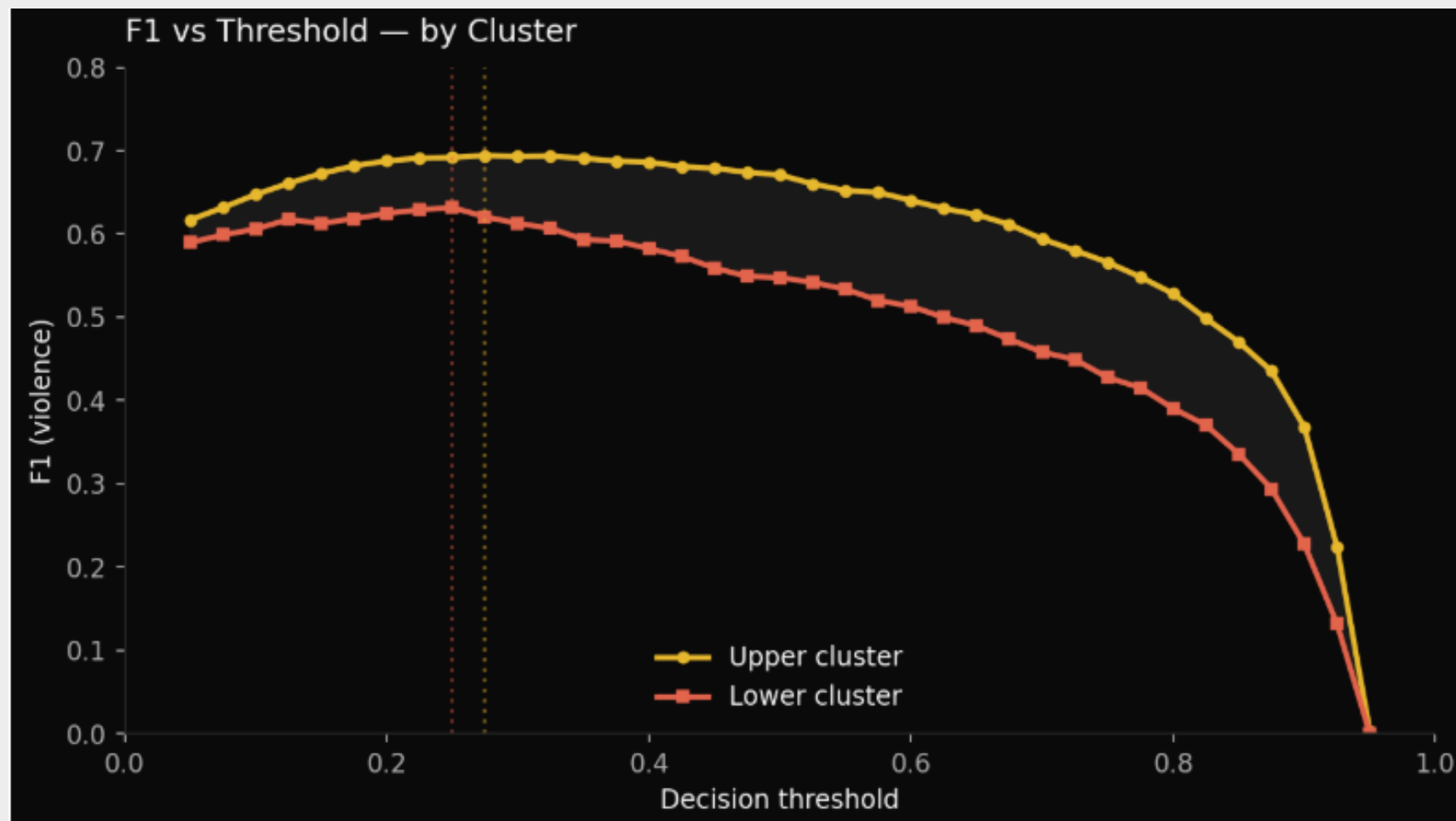
Both drop together — not model conservatism  
but representation mismatch with lower-cluster <sup>44</sup>  
violence style.



# Threshold Tuning

A real partial fix — but a representation-level deficit remains.

## | Threshold Tuning



## Threshold Tuning Works:

**Gap halves at  $t=0.25$**

Upper - F1 0.671  $\rightarrow$  0.694 (+0.023)

Lower - F1 0.547  $\rightarrow$  0.632 (+0.085)

Gap: 0.124  $\rightarrow$  0.062

## What remains:

**$\Delta = 0.062$  — a representation deficit & data limitations**

Threshold tuning fixes calibration — not what the model has learned.



# Reference

- [Google for DevelopersMediaPipe Solutions guide | Google AI Edge | Google for ...](#)
- [GitHubmediapipe/mediapipe at master · google-ai-edge/mediapipe](#)
- <http://ors.kmr.or.kr/guide/case/classstandards.do>
- [https://www.interdigital.com/data\\_sets/violent-scenes-dataset](https://www.interdigital.com/data_sets/violent-scenes-dataset)
- <https://kili-technology.com/blog/video-annotation#3d-cuboid-annotation>
- Glazner, N., Tsfaty, N., Shalev, S., & Weizman, A. (2026). Find the leak, fix the split: Cluster-based method to prevent leakage in video-derived datasets. arXiv. <https://doi.org/10.48550/arXiv.2511.13944>

**THANK YOU**